Sprint 4 · Reto «Herramientas internas»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. El equipo de reservas de TurisData calcula cada semana sus cifras **a mano en una hoja de cálculo**: es lento y cada persona lo hace distinto. Os pide una **herramienta interna**, un programa en Python que lea el fichero de reservas, calcule **tres KPI** (cifras que resumen cómo va el negocio: la ocupación, el ingreso medio por reserva y la tasa de cancelación por canal), **avise con claridad si el fichero está mal** y guarde el resultado en un fichero JSON que otros sistemas puedan leer.

Cómo se trabaja. No hace falta experiencia previa: os damos una **plantilla casi hecha** (funciones vacías y pruebas que os dicen si lo que escribís es correcto) y vuestro trabajo es rellenarla misión a misión. La ocupación será solo un *proxy* (una medida aproximada), porque el fichero no trae la capacidad real del hotel. El reto se hace en **seis misiones** repartidas en dos días, cada una con su paquete de ayuda (unas lecciones del curso «Python desde cero» y unos puzles del Taller de bloques) justo antes de necesitarlo: el **martes** la lógica con los datos en memoria y el **miércoles** los ficheros, los errores y la entrega. En clase tenéis unos **185 minutos de misiones el martes** (más 40 en la tutoría para acabar la misión 3) y **150 el miércoles** (misiones 4 y 5, README y entrega); lo que no quepa va a la tutoría o a casa. Cada persona hace además dos *katas* individuales de 10 minutos. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python 3.11 (solo biblioteca estándar: `csv`, `json`, `argparse`, `datetime`)	Escribir y ejecutar `kpis.py`	Ya instalado en el aula; si no, el de Colab
Editor y terminal: VS Code (recomendado) o Colab	Editar el código y ejecutar `python test_kpis.py`	En Colab, subid la carpeta y poned `!` delante: `!python test_kpis.py`
La plantilla `reto_herramientas/`	`kpis.py` con funciones vacías y pasos, `test_kpis.py` (por fases), `README_PLANTILLA.md`, `plantilla_kpis_pandas.py` (extra) y los archivos de las misiones 0 a 2 (`demo_cliente.py`, `reservas_demo.py`, `mision_1_datos.py`, `mision_2_decidir.py`)	En GitHub: *Code* → *Download ZIP*. Trabajad en una copia
Las tarjetas de misión `tarjetas_de_mision.md`	Tenerlas abiertas mientras trabajáis: qué hacer, con qué ayuda y cuándo está hecha cada misión	En el repositorio del curso
El curso «Python desde cero» (abrir) y **el Taller de bloques** (abrir)	El paquete de ayuda de cada misión (lecciones mínimas y puzles) y las dos katas	En el navegador, sin instalar nada; podéis exportar vuestro progreso
El CSV real `reservas_turisdata.csv`	6000 reservas **sintéticas**	Botón *Download raw file*; dejadlo **junto a `kpis.py`**
Cuadernos del sprint: `S04_03` y `S04_04`	Las pistas para las funciones (diccionarios, CSV, JSON, excepciones)	Colab; su código explicado
Git y GitHub + cuaderno `S04_05`	Guardar versiones (*commits*) y entregar	Terminal o VS Code. Si falla, subid los ficheros desde la web (*Add file* → *Upload files*): cada subida es un commit
Cómo entregar en GitHub	Subir la carpeta `sprint-04/`	Guía de entrega

Recurso	Para qué lo usaréis	Dónde está
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-04/`: 4 ficheros del equipo (`kpis.py`, `test_kpis.py`, `README.md`, `salida.json`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **4 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 7). «El reto» no es un fichero aparte: son esos cuatro ficheros. Además, el CSV va a `data/` si aún no está, y los extras solo si los hacéis. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-04/`**, salvo el CSV.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`kpis.py`	**Lo modificáis.** Os damos una plantilla con las funciones vacías (`calcular_kpis` y la estructura de `main` ya vienen hechas)	Escribís el código de las funciones sobre la propia plantilla	**Sí** · `sprint-04/`
`test_kpis.py`	**Ya hecho.** Pruebas completas, que se ejecutan por fases (`python test_kpis.py 1`, 2 o sin número)	Nada: se copia tal cual y **no se modifica**	**Sí**, sin cambios · `sprint-04/`
`README.md`	**Lo creáis a partir de una plantilla.** Os damos `README_PLANTILLA.md` con huecos	Copiarla como `README.md` y rellenarla	**Sí** · `sprint-04/`
`salida.json`	**Lo genera el programa** al ejecutarlo con el CSV real	Ejecutar el programa; no se escribe a mano	**Sí** · `sprint-04/`
`reservas_turisdata.csv`	**Ya hecho.** Los datos reales (sintéticos)	Nada: se copia junto a `kpis.py` sin cambiarlo	**Sí**, sin cambios · `data/` (si aún no está)
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-04/`
`kpis_pandas.py` (extra)	**Lo modificáis.** Parte de `plantilla_kpis_pandas.py`	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
Pruebas propias (extra)	**Lo creáis vosotros**, en un fichero aparte	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
`demo_cliente.py` y `reservas_demo.py`	**Ya hechos.** La demostración del cliente y sus 8 reservas de ejemplo	Ejecutarlos y leerlos (misión 0)	**No se suben**
`mision_1_datos.py` y `mision_2_decidir.py`	**Lo modificáis.** Ejercicios con huecos (`TODO`) y comprobación automática	Rellenar cada `TODO` (misiones 1 y 2)	**No se suben**
Copias de prueba de la misión 5 (`sin_col.csv`…)	**Las creáis** para romper el programa a propósito	Usarlas y borrarlas	**No se suben**

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Leer el CSV	`leer_reservas`: convierte los tipos y **descarta y cuenta** las filas defectuosas (noches < 1, precio ≤ 0, `cancelada` distinto de 0/1, datos que no se pueden convertir)	Mínimo	`kpis.py`
Tasa de cancelación por canal	canceladas ÷ reservas del canal × 100, con 1 decimal	Mínimo	`kpis.py` · `tasa_cancelacion_por_canal`
Ingreso medio por reserva	media de `noches × precio_noche` de las reservas **no canceladas**, con 2 decimales (`0.0` si no hay)	Mínimo	`kpis.py` · `ingreso_medio_por_reserva`
Ocupación (proxy)	noches no canceladas ÷ (días del periodo × habitaciones) × 100; el periodo va de la primera a la última llegada. En el mínimo, con ayuda del docente	Estándar	`kpis.py` · `ocupacion_proxy`
Guardar el resultado	`exportar_json` y `main`: JSON en UTF-8 con los tres KPI y la sección `calidad` (filas válidas, descartadas y avisos). En el mínimo, con ayuda del docente	Estándar	`kpis.py` → `salida.json`
Avisar de los errores	Fichero inexistente o columna que falta → mensaje claro y código de salida 1, sin `Traceback`	Mínimo	`kpis.py` · `main` y `leer_reservas`
Resultado con el CSV real	`python kpis.py reservas_turisdata.csv salida.json --capacidad 40`	Mínimo	`salida.json`
Pruebas	`python test_kpis.py` pasa todo. **No se modifican** las dadas; las vuestras, aparte	Estándar	`test_kpis.py`
Código limpio	Docstring en cada función y estilo PEP 8	Estándar	`kpis.py`
README	Cómo se ejecuta y se prueba · qué es cada KPI · por qué la ocupación es un proxy · uso de la IA (qué pedisteis y qué revisasteis) · la revisión con la rúbrica. En el mínimo, versión corta	Mínimo	`README.md`
Git	Al menos 3 *commits* con mensaje claro, de personas distintas, sin claves ni datos personales	Mínimo	Historial del repositorio del equipo
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el código y el README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-04/`
Extras	Pruebas propias · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	Extra	`test_kpis.py` · `kpis_pandas.py` · `kpis.py`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay código, pero fallan muchas pruebas o no se ejecuta con el CSV real	Pasan las pruebas de lectura, cancelación e ingreso medio; sale `salida.json` con el CSV real; un error de fichero o de columna da mensaje y código 1	`python test_kpis.py` da **12 de 12**; la ocupación y la sección `calidad` salen bien; cada función tiene docstring y el estilo es PEP 8	Además, pruebas propias que pasan (lista vacía, división entre cero…) o `kpis_pandas.py` con los mismos resultados

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en el ordenador de quien lo escribió	El README tiene los comandos, pero faltan datos (carpeta, versión de Python…)	Otra persona obtiene **el mismo JSON y las pruebas en verde** siguiendo solo el README; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, el README indica versión de Python, estructura de carpetas y el comando exacto
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	El README no explica qué es cada KPI	Define los tres KPI, pero no sus límites	Define cada KPI **y explica por qué la ocupación es un proxy** (qué capacidad se supuso), sin jerga	Además, explica cómo cambia la ocupación si cambia la capacidad, o qué conclusión saca el cliente del resultado
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No dice si se ha usado IA	Dice que se ha usado IA, sin detalle	Dice **qué se pidió, qué se revisó y qué se cambió**, y quien escribe sabe explicar cualquier línea	Además, cuenta un fallo concreto de la IA que detectaron las pruebas, y menciona que los datos son sintéticos y no hay datos personales

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar**, en una carpeta limpia, `python test_kpis.py` y `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`, copiando solo lo que dice el README. Anotar si algo no funciona.
2. **Leer el README como si fuera el cliente**: ¿se entiende qué hace cada KPI y por qué la ocupación es un proxy? Subrayar lo que no se entienda.
3. **Mirar el código**: ¿tiene docstring cada función? ¿Hay algo que no sepáis explicar? Que quien lo escribió lo explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · 12 de 12 pruebas, pero falta el docstring de `main`»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso: seis misiones

El reto se hace en **seis misiones**, en **dos fases**: el **martes** la lógica con las reservas ya **en memoria** (sin ficheros) y el **miércoles** los ficheros, los errores, el JSON y la entrega. Cada misión dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecha**. Cada una trae su **paquete de ayuda** (unas pocas lecciones del curso «Python desde cero» y unos puzles del Taller de bloques) **justo antes de necesitarlo**. Las ayudas (IA, errores, atascos) están al final. Para tenerlas abiertas mientras trabajáis, usad las tarjetas de misión.

Cómo se reparte el trabajo. En **parejas de piloto y copiloto**: el piloto teclea, el copiloto lee en voz alta lo que pide la misión y comprueba; **cambiad de papel en cada misión**. En un equipo de 3, la tercera persona revisa (ejecuta las pruebas y lee el docstring). Al final de cada día, **cada persona sola** escribe una **kata**: una función corta, sin ayuda, en el Taller. Es lo que asegura que **cada persona** ha aprendido, no solo el equipo.

Misión	Cuándo	Qué se construye	Paquete de ayuda	Se comprueba con
0 · Mirar la demo del cliente	día 1 · 35 min	Entender el programa del cliente	Curso `type()` y f-strings	Cinco preguntas de lectura
1 · Datos en memoria	día 1 · 50 min	Leer y modificar una lista de reservas	Curso listas y diccionarios · Taller, bloque 1	`python mision_1_datos.py`
2 · Decidir y repetir	día 1 · 50 min	Los contadores de los KPI (`if`, `for`)	Curso `if/elif/else` y `for`	`python mision_2_decidir.py`
3 · Funciones que calculan	día 1 · 50 + 40 min	`tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva` (y `ocupacion_proxy` con el docente)	Curso funciones · Taller, bloque 2	`python test_kpis.py 1`
Kata 1 (individual)	día 1 · 10 min	`noches_confirmadas`	Lo hecho en la misión 3	El propio Taller

Misión	Cuándo	Qué se construye	Paquete de ayuda	Se comprueba con
4 · Ficheros	día 2 · 50 + 50 min	`_convertir_fila` y `leer_reservas`	Curso excepciones y ficheros · Taller, bloque 3	`python test_kpis.py 2`
5 · Salida	día 2 · 25 + 25 min	`exportar_json`, `main`, README y entrega	Taller, bloque 4	`python test_kpis.py` (12 de 12)
Kata 2 (individual)	día 2 · 10 min	`media_precio`	Lo hecho en la misión 4	El propio Taller

FASE 1 · MARTES · la lógica, con las reservas en memoria

Misión 0 · Mirar la demostración del cliente (día 1, 35 minutos)

1. Preparar la carpeta de trabajo con la plantilla y el CSV.
2. Ejecutar `python demo_cliente.py` y leer el programa de arriba abajo.
3. Responder, sin mirar el ordenador, a cinco preguntas de lectura.
4. Hacer en el curso `type()` y los f-strings (ejercicios mínimos).

Cómo se hace (unos 15 minutos para los puntos 1 a 3 y 20 para el curso)

- **Descargar.** Opción rápida: en el repositorio del curso pulsad *Code* → *Download ZIP*, descomprimid y copiad a una carpeta nueva (por ejemplo `sprint-04`) **todo el contenido** de `sprints/S04/3_Reto/4_Plantillas_y_entrega/reto_herramientas/` (`kpis.py`, `test_kpis.py`, `README_PLANTILLA.md`, `demo_cliente.py`, `reservas_demo.py`, `mision_1_datos.py`, `mision_2_decidir.py` y `plantilla_kpis_pandas.py`) y, de `datos/`, `reservas_turisdata.csv`. Todo **junto, en la misma carpeta**.
- **Abrir la terminal.** En VS Code: *File* → *Open Folder…* (elegid `sprint-04`) y *Terminal* → *New Terminal*. Si el aula no tuviera Python, usad Colab: subid los ficheros al panel izquierdo y poned `!` delante de cada comando.
- **Comprobar Python.** Escribid `python --version`; debe salir 3.11 o superior. Si da error, probad `python3 --version` y usad `python3` en lugar de `python` en todo el reto.
- **Ejecutar la demostración.** `python demo_cliente.py`. Los 8 datos de ejemplo están en `reservas_demo.py`: no hace falta tocarlos.
- **Preguntas de lectura.** (1) ¿Cuántas reservas analiza el programa y qué línea lo calcula? (2) ¿Por qué `canceladas` empieza en `0` y `por_canal` en `{}`? (3) Si la primera reserva pasara a estar cancelada, ¿qué cifras cambiarían? (4) ¿Qué hace `:.1f` dentro de las llaves del f-string? (5) Si RESERVAS estuviera **vacía**, ¿qué error saldría y en qué línea? (probadlo en una copia).
- **Curso (20 minutos).** `type()` (los 3 ejercicios) y formateo de textos (ejercicios 1 y 2). El docente hace el primer ejercicio de cada lección en pantalla.

Qué deberíais ver. Al ejecutar la demostración:

```
=== Resumen semanal de reservas ===
Reservas analizadas: 8
Canceladas: 3 (37.5 %)
Ingreso medio por reserva confirmada: 182.00 €
  - directo: 3 reservas
  - agencia: 2 reservas
  - web_intermediaria: 3 reservas
```

Hecho cuando: cada persona sabe responder con sus palabras las cinco preguntas. No hace falta entender cada línea todavía: la idea es ver que **un programa de verdad son cuatro piezas** (datos, bucle, `if` y mensaje).

Misión 1 · Los datos en memoria (día 1, 50 minutos)

1. Hacer en el curso listas y diccionarios (ejercicios mínimos).
2. Completar `mision_1_datos.py`.
3. Montar con piezas los dos primeros puzles del bloque 1 del Taller.

Cómo se hace

- **Curso (25 minutos).** Listas (ejercicios 1 y 2) y diccionarios (los 3 ejercicios).
- **El archivo (15 minutos).** Abrid `mision_1_datos.py`: una reserva es un diccionario y muchas reservas, una lista de diccionarios. Sustituid cada `TODO` y ejecutad `python mision_1_datos.py`. Pistas: `len(...)`, `reservas[0]["canal"]`, `reservas[-1]` (la última), `reservas.append({...})` y un f-string.
- **Taller (10 minutos).** «El total de una reserva» y «Un mensaje con f-string».

Qué deberíais ver. Al empezar, `Misión 1: 0 de 6 apartados bien` con la lista de lo que falta (es lo normal). Cada apartado que arregláis lo quita de la lista. Si sale `KeyError` o `IndexError`, leed qué clave o posición no existe.

Hecho cuando: `python mision_1_datos.py` termina con **« ¡Misión 1 superada!»**.

Misión 2 · Decidir y repetir (día 1, 50 minutos)

1. Hacer en el curso `if/elif/else` y `for` (ejercicios mínimos).
2. Completar `mision_2_decidir.py` con los contadores.

Cómo se hace

- **Curso (25 minutos).** `if / elif / else` (ejercicios 1 y 2) y `for` (los 3 ejercicios).
- **El archivo (25 minutos).** Con las 8 reservas de `reservas_demo.py`: contar canceladas, sumar las noches de las no canceladas, contar reservas por canal (con un diccionario) y listar los canales de las confirmadas de 3 noches o más. **El patrón de contar es siempre el mismo:** una variable que empieza en `0`, un `for` que recorre las reservas y un `if` que suma 1 cuando toca. Para contar **por canal**: `por_canal[canal] = por_canal.get(canal, 0) + 1` (mirad cómo lo hace `demo_cliente.py`).

Qué deberíais ver. `Misión 2: N de 4 apartados bien` y los que faltan. Los resultados esperados: 3 canceladas, 12 noches confirmadas, `{'directo': 3, 'agencia': 2, 'web_intermediaria': 3}` y `['agencia', 'directo']`.

Hecho cuando: `python mision_2_decidir.py` termina con **« ¡Misión 2 superada!»**. Esto es **la base de todos los KPI**: lo que viene ahora es meterlo dentro de funciones.

Misión 3 · Funciones que calculan (día 1, 50 minutos + 40 minutos después de la consolidación)

1. Hacer en el curso funciones (`def`, parámetros y `return`).
2. Montar con piezas las dos funciones del bloque 2 del Taller.
3. Escribir en `kpis.py` `tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva`, con sus pruebas.
4. Ver en pantalla, con el docente, `ocupacion_proxy`.

Cómo se hace

- **Curso (10 minutos).** Funciones (ejercicios 1 y 2).
- **Taller (10 minutos).** «Contar con un diccionario» e «Ingreso medio de las reservas confirmadas»: **son casi vuestras dos funciones**, montadas con piezas.
- **Las pruebas.** Abrid `kpis.py` y ejecutad `python test_kpis.py 1`: la fase 1 son 4 pruebas y empiezan fallando. Programad **una función cada vez**, siguiendo los comentarios «Paso» de su docstring: ejecutad la prueba, mirad qué dice que esperaba y qué recibió, corregid y repetid. Es el método de los profesionales: trabajar **guiados por las pruebas**. **Nunca cambiéis `test_kpis.py` para que pase.**
- **Qué cuenta como `return`.** `return` **devuelve** el valor; `print` solo lo enseña, y las pruebas necesitan el valor.

3.1 · `tasa_cancelacion_por_canal` (25 minutos)

Recibe y devuelve. La lista de reservas → un diccionario `{canal: porcentaje}`. Con 3 reservas del canal `web` y 1 cancelada: `{"web": 33.3}`. Sin reservas: `{}`.

Orden	Qué hacéis	Pista de Python
1	Crear dos diccionarios vacíos: reservas totales por canal y canceladas por canal	`total = {}` y `canceladas = {}`
2	Recorrer las reservas: sumar 1 al total de su canal y, si `cancelada` vale 1, sumar 1 a las canceladas	`total[c] = total.get(c, 0) + 1` empieza un contador
3	Por cada canal, calcular el porcentaje con 1 decimal	`round(100 * canceladas[c] / total[c], 1)`

Se comprueba con `test_tasa_cancelacion…` (incluye el redondeo). **Pista de cuaderno:** `S04_03` (diccionarios).

3.2 · `ingreso_medio_por_reserva` (20 minutos)

Recibe y devuelve. La lista de reservas → un número con 2 decimales. Con reservas confirmadas de 2×100 €, 4×50 € y 1×120 €: (200 + 200 + 120) ÷ 3 = **173.33**. Sin ninguna confirmada: `0.0` (sin dividir entre cero).

Orden	Qué hacéis	Pista de Python
1	Hacer una lista con `noches × precio_noche` **solo** de las reservas no canceladas	Comprensión de lista con `if r["cancelada"] == 0`
2	Si la lista está vacía, devolver `0.0`	`if not lista:`

Orden	Qué hacéis	Pista de Python
3	Si no, la media con 2 decimales	`round(sum(lista) / len(lista), 2)`

Se comprueba con `test_ingreso_medio`…. **Pista de cuaderno:** `S04_03` (comprensiones de lista).

3.3 · `ocupacion_proxy` (10 minutos, con el docente)

Recibe y devuelve. La lista de reservas y la capacidad (n° de habitaciones) → un porcentaje con 1 decimal. Ejemplo de la prueba: 7 noches confirmadas, un periodo de 10 días y 5 habitaciones → 7 ÷ (10 × 5) × 100 = **14.0**.

Orden	Qué hacéis	Pista de Python
1	Si la capacidad es ≤ 0, dar un error (no se puede dividir entre cero)	`raise ValueError("La capacidad debe ser mayor que 0")`
2	Si no hay reservas, devolver `0.0`	`if not reservas:`
3	Calcular los días del periodo: de la primera a la última llegada, **contando también el primer día**	`(max(fechas) - min(fechas)).days + 1`
4	Sumar las noches de las reservas **no canceladas**	`sum(...)` sobre una comprensión con `if`
5	Aplicar la fórmula con 1 decimal	`round(100 * noches / (dias * capacidad), 1)`

Se comprueba con `test_ocupacion_proxy`. En el nivel mínimo, esta función se completa con ayuda del docente. Recordad que es un **proxy**: se explica en el README.

Qué deberíais ver. Al terminar la misión, `python test_kpis.py 1` termina con `4 de 4 pruebas superadas`.

Hecho cuando: pasan las cuatro pruebas de la fase 1 (si `ocupacion_proxy` la ha hecho el docente, la habéis **copiado y explicado** con vuestras palabras).

Al final del día · Kata 1 (10 minutos, **cada persona sola**). Abrid la kata 1 y escribid `noches_confirmadas(reservas)` sin mirar nada. Apuntad cuántas pistas usáis. Si no sale, no pasa nada: se repite mañana.

FASE 2 · MIÉRCOLES · ficheros, errores y entrega

Misión 4 · Leer el CSV sin que se rompa (día 2, 50 minutos + 50 minutos)

1. Hacer en el curso excepciones y ficheros (ejercicios mínimos) y los tres puzles del bloque 3 del Taller.
2. Programar `_convertir_fila` en `kpis.py`.
3. Programar `leer_reservas` y ver pasar las pruebas de lectura.

Cómo se hace

- **Curso y Taller (50 minutos).** Excepciones (ejercicios 1 y 2) y ficheros de texto, CSV y JSON (ejercicios 2 y 3), 30 minutos; después, «Leer un CSV sin que se rompa», «¿Qué error es cada uno?» y «Guardar los KPIs en JSON», 20 minutos.
- **El trabajo (50 minutos).** En `kpis.py`, primero `_convertir_fila` y después `leer_reservas`, con los comentarios «Paso». Ejecutad `python test_kpis.py 2` después de cada una.
- **Si a las 09:55 todavía no habéis empezado `leer_reservas`:** avisad al docente. La proyecta resuelta, **la copiáis y la explicáis con vuestras palabras, línea a línea**, y seguís con la misión 5. Es una ayuda legítima, y la declaráis en el README («Uso de IA y límites»).

4.1 · `_convertir_fila` y `leer_reservas` (50 minutos, todo el equipo, pilotos y copilotos rotando cada 15)

Recibe y devuelve. `_convertir_fila` recibe una fila con todo en texto, `{"fecha_llegada": "2025-01-01", "noches": "2", "precio_noche": "100.0", "canal": "directo", "cancelada": "0"}`, y la devuelve con los tipos buenos: la fecha como `date`, `noches` como `int`, `precio_noche` como `float` y `cancelada` como `int`. `leer_reservas(ruta)` devuelve **dos cosas**: la lista de reservas y la lista de avisos.

Cómo pensarlo, en este orden (así las pruebas se van poniendo en verde una a una):

Orden	Qué hacéis	Pista de Python	Prueba que pasa a verde
1	Convertir los tipos en `_convertir_fila`	`int()`, `float()` y `date.fromisoformat()` ya lanzan `ValueError` si el texto no sirve	`test_leer_reservas_convierte_tipos`

Orden	Qué hacéis	Pista de Python	Prueba que pasa a verde
2	Validar los valores: además del tipo, el valor debe tener sentido (noches ≥ 1, precio > 0, `cancelada` 0 o 1, canal no vacío)	`raise ValueError("noches debe ser >= 1")`	`test_valores_imposibles…`
3	Leer el fichero en `leer_reservas`; si no existe, lanzar un error que **incluya la ruta**	`open(ruta, newline="", encoding="utf8")`, `csv.DictReader` y `except FileNotFoundError`	`test_fichero_inexistente`
4	Comprobar la cabecera: si falta una columna, un error que **nombre las que faltan**	Comparar `lector.fieldnames` con `COLUMNAS_OBLIGATORIAS`	`test_faltan_columnas`
5	Descartar y avisar: probar cada fila, y si falla, guardar el aviso y seguir	`for n, fila in enumerate(lector, start=2)` (la fila 1 es la cabecera), `try/except ValueError` y `f"Fila {n} descartada: {error}"`	`test_filas_defectuosas…`
6	Devolver **dos cosas a la vez**: las reservas buenas y los avisos	`return reservas, avisos` (una *tupla*: dos valores juntos). Quien llama los recoge con `reservas, avisos = leer_reservas(ruta)`	`test_leer_reservas_convierte_tipos`

Ejemplo. La fila 3 del fichero tiene `noches = 0` → aviso `Fila 3 descartada: noches debe ser >= 1`, y esa fila no entra en la lista de reservas. **Pista de cuaderno:** S04_04 (CSV, JSON y `try/except`).

Qué deberíais ver. `python test_kpis.py 2` da `7 de 8 pruebas superadas`: solo falla `test_exportar_json_y_main`, que es de la misión 5. (Si no habéis hecho el nivel extra de pandas, esa prueba se omite y cuenta como superada.)

Hecho cuando: pasan las pruebas de lectura (`test_leer_reservas_convierte_tipos`, `test_fichero_inexistente`, `test_faltan_columnas`, `test_filas_defectuosas…` y `test_valores_imposibles…`) y la de los datos reales.

Misión 5 · Guardar el resultado, ponerlo en marcha y entregar (día 2, 50 minutos)

1. Montar con piezas `main` en el Taller (bloque 4).
2. Programar `exportar_json` y `main`, y ejecutar el programa con el CSV real.
3. Provocar tres errores a propósito.
4. README, revisión con la rúbrica y subida a `sprint-04/`.

Cómo se hace (10 minutos de Taller y 15 de programar; 10 de errores; 25 de README y entrega)

- **Taller.** «Encadena las funciones» y «Escribe la función principal»: es exactamente vuestro `main`.
- **Programar y probar**, en este orden: 5.1 (exportar y ejecutar), 5.2 (provocar errores) y 5.3 (README y entrega).

5.1 · Exportar el resultado y ejecutar el programa entero (15 minutos)

1. Programar `exportar_json` y `main`.
2. Ejecutar el programa con el CSV real.
3. Abrir `salida.json` y compararlo con el resultado esperado.

Cómo se hace

- **`exportar_json(datos, ruta)`**: abrid `ruta` para escribir con `encoding="utf8"` y guardad con `json.dump(datos, f, ensure_ascii=False, indent=2)`. Sin `ensure_ascii=False` las tildes saldrían como `\u00f3`. Pista: S04_04.
- **`main(argumentos=None)`**: la plantilla ya lee los argumentos (`args.entrada`, `args.salida` y `args.capacidad`). Vosotros, **dentro de un `try`**: (1) `leer_reservas(args.entrada)`; (2) `calcular_kpis(reservas, args.capacidad)`, que ya viene hecha; (3) añadir al resultado la clave `"calidad"` con `filas_validas`, `filas_descartadas` y `avisos` (si hay muchos avisos, guardad solo los 20 primeros); (4) `exportar_json`; (5) imprimir un mensaje de éxito y **devolver 0**. En el `except (FileNotFoundError, ValueError)`: imprimir `ERROR: …` **por `sys.stderr`** y **devolver 1**.
- **Ejecutar:** `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`

Qué deberíais ver. Por pantalla, un mensaje como `OK: 6000 reservas analizadas, 0 descartadas -> salida.json` y, en la carpeta, un fichero `salida.json` con este contenido:

```
{
  "reservas_analizadas": 6000,
  "ocupacion_proxy_pct": 42.0,
  "ingreso_medio_por_reserva": 576.04,
  "tasa_cancelacion_por_canal_pct": {"empresa": 43.3, "directo": 31.5, "web_intermediaria": 53.9, "agencia": 44.0},
  "calidad": {"filas_validas": 6000, "filas_descartadas": 0, "avisos": []}
}
```

Cuando lo tengáis: existe `salida.json` con esas cifras y `python test_kpis.py` da `12 de 12 pruebas superadas` (con `kpis_pandas.py` ausente, la prueba opcional se omite y cuenta como superada). Si las cifras son distintas, mirad la «Ayuda 2».

5.2 · Probar que no se rompe (10 minutos)

1. Ejecutar el programa con un fichero que no existe.
2. Quitar una columna al CSV (en una copia) y ejecutar de nuevo.
3. Poner un dato incorrecto en una fila (en otra copia) y ejecutar.
4. Comprobar en cada caso que el mensaje es claro, sin el texto rojo de `Traceback`.

Cómo se hace. Trabajad siempre sobre **copias**: el CSV original no se toca.

Caso	Cómo se provoca	Qué deberíais ver	Código de salida
1. Fichero que no existe	`python kpis.py no_existe.csv salida.json`	`ERROR: No se encuentra el fichero: no_existe.csv`	`1`
2. Falta una columna	Copiad el CSV como `sin_col.csv` y, con el editor, cambiad en la primera línea `precio_noche` por `precio`. Ejecutad con `sin_col.csv`	`ERROR: Faltan columnas obligatorias: ['precio_noche']`	`1`
3. Dato incorrecto	Copiad el CSV como `con_error.csv` y, con el editor, escribid `abc` como precio de la fila 5 y `0` como noches de la fila 10. Ejecutad con `con_error.csv`	El programa **no se rompe**: `OK: 5998 reservas analizadas, 2 descartadas -> …` y en `calidad.avisos` aparecen `Fila 5 descartada: …` y `Fila 10 descartada: noches debe ser >= 1`	`0`

El código de salida se mira justo después de ejecutar: en Linux o Mac, `echo $?`; en Windows (cmd), `echo %errorlevel%`; en PowerShell, `echo $LASTEXITCODE`.

Al terminar, borrad las copias de prueba (`sin_col.csv`, `con_error.csv`, el JSON de prueba): **no se suben**.

Cuando lo tengáis: los tres casos se comportan así. Ese será vuestro «provocar un error» en la review.

5.3 · README y entrega (25 minutos)

1. Copiar `README_PLANTILLA.md` como `README.md` y rellenarlo.
2. Pedir a alguien que no escribió esa parte que lo revise con la rúbrica y anote el resultado en el README.
3. Subir los ficheros a `sprint-04/` en tres o más subidas, hechas por personas distintas.

Cómo se hace

- **El README.** Cada sección de la plantilla pide una cosa. Para el **nivel mínimo** bastan estas cuatro, en pocas líneas:

Sección de la plantilla	Qué escribís
Qué hace	Dos o tres frases: lee el CSV de reservas, calcula tres KPI, avisa si el fichero está mal y guarda un JSON
Cómo se ejecuta y cómo se prueban	Los comandos `python kpis.py reservas_turisdata.csv salida.json --capacidad 40` y `python test_kpis.py`, y el resultado (`12 de 12 pruebas superadas`)
KPIs y decisiones	Qué es cada KPI y **por qué la ocupación es un proxy**. Ejemplo: «No sabemos cuántas habitaciones tiene el hotel: suponemos 40 y, con 20, saldría el doble. Además contamos todas las noches el día de llegada»
Uso de IA y límites	Qué le pedisteis a la IA y qué revisasteis; los datos son sintéticos

Para el **nivel estándar**, completad también «Requisitos», «Gestión de errores» (qué errores controla, qué mensaje da y que devuelve código 1) y la versión con pandas si la hacéis.

- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y rellena la sección «Revisión con la rúbrica» del final del README: un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos». Si sobran minutos, empezad el extra (pruebas propias o `plantilla_kpis_pandas.py`).
- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-04`**; (2) *Add file* → *Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `kpis.py` («Añado lectura del CSV y KPI»), la B sube `test_kpis.py` y `salida.json` («Añado pruebas y resultado»), la C sube `README.md` («Añado README»). Los nombres de archivo, **en minúsculas y sin espacios**. El CSV va a `data/` del repositorio solo si aún no está.
- **Alternativa con Git en el terminal** (si ya lo habéis practicado en S04_05): `git add kpis.py test_kpis.py README.md salida.json`, `git commit -m "Añado lectura del CSV y KPI"` y `git push`. Cada persona lo hace con su propia cuenta.

Qué deberíais ver. En la carpeta `sprint-04` del repositorio, los cuatro ficheros del equipo y una hoja de autoevaluación por persona (**4 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos.

Cuando lo tengáis: otra persona podría ejecutar el proyecto siguiendo solo el README, la sección «Revisión con la rúbrica» está rellena y los tres *commits* están en el repositorio.

Después, 15 minutos de entrega y review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-04/`, y el equipo enseña, en 3–5 minutos, el script con el CSV real, el JSON, un error provocado con su mensaje y las pruebas en verde. Explicad una decisión de diseño, por ejemplo cómo definisteis la ocupación.

Qué deberíais ver. `python test_kpis.py` da `12 de 12 pruebas superadas`; en `sprint-04/`, los cuatro ficheros del equipo y una hoja de autoevaluación por persona.

Hecho cuando: existe `salida.json` con las cifras esperadas, las 12 pruebas pasan, los tres errores provocados dan su mensaje, y el README, la revisión y los *commits* están subidos.

Al final del día · Kata 2 (10 minutos, **cada persona sola**). Abrid la kata 2 y escribid `media_precio(ruta)`, que lee un CSV ignorando las filas defectuosas. Apuntad cuántas pistas usáis.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas mandan:** si el código de la IA no las pasa, no vale.
- **Declaradlo** en la sección «Uso de IA y límites» del README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un error	«Me sale este error: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que lo entendéis y lo arregláis vosotros
Entender una función	«Explícame línea por línea qué hace esta función: *[código]*»	Ejecutadla con un ejemplo pequeño
Pedir una pista	«Quiero calcular el % de cancelaciones por canal con la biblioteca estándar. ¿Qué estructura me conviene? No me des el código»	Que respeta «solo biblioteca estándar»
Revisar el estilo	«¿Sigue PEP 8 este código? ¿Qué cambiarías?»	Que las pruebas siguen pasando

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`ModuleNotFoundError: kpis` (o `reservas_demo`)	Se ejecuta desde otra carpeta	Abrir la terminal **dentro** de la carpeta del reto
La prueba dice que la función devuelve `None`	La función imprime con `print` pero no termina con `return`	Terminar con `return valor`: devolver no es lo mismo que mostrar
La media sale rara	Se incluyen reservas canceladas en el ingreso	**Solo las no canceladas**
La ocupación no sale 42.0	Falta el `+ 1` en los días, se cuentan canceladas o la capacidad no es 40	Releer la fórmula y ejecutar con `--capacidad 40`

Error	Por qué ocurre	Cómo evitarlo
Las pruebas de la fase 2 fallan todas a la vez	Dependen de `leer_reservas`	Empezar por esa función; la fase 1 (`python test_kpis.py 1`) no depende de ella
El programa se rompe con un fichero mal	No se controlan las excepciones	`try` / `except` con tipos concretos y mensajes claros
Tildes rotas	Falta `encoding="utf8"`	Añadirlo al abrir y escribir ficheros
Cambiar las pruebas para que pasen	Prisa	No tocar las pruebas dadas; añadir las propias aparte
Un solo *commit* gigante	«Lo subo al final»	Commits pequeños mientras avanzáis

Ayuda 3 · Si os atascáis

1. Leed el mensaje de la prueba que falla: dice qué esperaba y qué recibió. Probad la función en una celda con un ejemplo pequeño.
2. Repasad el apartado de esa función en las misiones 3 y 4, la lección del curso y su puzle del Taller.
3. Preguntad al equipo y, después, al docente.